

TP Data Structure

Compilation

```
gcc *.c  
./a.out
```

Complexités : tableau dynamique et liste chaînée

Opération	Dyn. Array	Linked List	Gagnant
get(index)	$O(1)$	$O(n)$	Array
find(value)	$O(n)$	$O(n)$	Égalité
insert_front	$O(n)$	$O(1)$	List
insert_back	$O(1)$ amorti	$O(n)$	Array
remove_front	$O(n)$	$O(1)$	List
Parcours complet	$O(n)$	$O(n)$	Égalité

Complexités : table de hachage

Opération	Moyenne	Pire cas
hash_insert	$O(1)$	$O(1)$
hash_contains	$O(1)$	$O(n)$
hash_free	$O(n + m)$	$O(n + m)$

n = nombre d'éléments, m = TABLE_SIZE

Remarques

Tableau dynamique : accès par indice immédiat, mais toute opération en tête oblige à décaler l'ensemble des éléments.

Liste chaînée : insertion et suppression en tête en temps constant, mais insert_back en $O(n)$ faute de pointeur vers la fin.

Table de hachage : recherche en temps constant tant que les valeurs sont bien réparties dans les buckets, sinon la structure dégénère en liste chaînée.